

# M8：运算符重载、流、命名空间与头文件

CSC3002 基础讲义 · 第 4 册 · 前置：M2 的函数模型、M6 的字符串、M7 的类对象

**核心问题：**怎样让自定义类型拥有自然的操作写法，并把接口和实现组织成可以编译的程序？学完应能把 `a+b`、`a+=b`、`cout<<a` 读回函数调用，说明返回值与返回引用的不同，并读懂 `namespace`、`.h` 和 `.cpp` 的分工。

路线：运算符也能调用函数 → `+` 与 `+=` 的契约 → 成员/非成员 → 返回引用 → 流与连续操作 → 命名空间与多文件 → `Direction` 的重载 → 文件、字符与字符串流 → A1 接口。采用 C++17；课件代码和自编教学版分别标明，E22 Q4/Q22 在最后完整讲解。

## 阅读依据与课程定位

这是对实际课程材料重新核对后的修订版，核对日期为 2026-10-08。L 后的数字是课件代号；“第几张”按文件实际幻灯片顺序计数，从封面开始，不沿用旧模板页脚。E22 页码为 PDF 页码。

- L2: 2-CSC3002\_Lecture Notes Functions and Libraries Part 1(2).pptx
- L3: 3-CSC3002\_Lecture Notes Functions and Libraries Part 2.pptx
- L4: 4-CSC3002\_Lecture Notes\_Strings.pptx
- L5: 5-CSC3002\_Lecture Notes Streams(3).pptx
- E22: CSC3002 Final 2022 Spring.pdf
- A1: assignment1.pdf
- T1: CSC3002\_Tutorial1\_Supplement.pdf

| 本册学习任务                                                 | 原材料定位                        | 要能独立完成的动作                         |
|--------------------------------------------------------|------------------------------|-----------------------------------|
| 函数重载、signature、运算符形式                                   | L2 第 17-18、33-41 张；E22 Q22   | 按参数列表判断，区别声明和调用                   |
| client、interface、implementation、多文件                    | L3 第 5-7、11-23 张；L4 第 18 张   | 把声明可见与定义链接分开                      |
| Direction 的 <code>&lt;&lt;</code> 与前后置 <code>++</code> | L3 第 14-19 张                 | 读出标记 <code>int</code> 与修改/返回契约    |
| 流参数、返回流、输入状态                                           | L5 第 10、16-18、26-32 张；E22 Q4 | 跟踪同一个流与目标对象                       |
| 文件流、字符串流与格式数据                                          | L5 第 20-39 张                 | 使用读取操作的成功状态控制循环                   |
| A1 接口与编译驱动                                             | A1 第 1-2 页；T1 第 1-2 页        | 保持命名空间和签名，正确构建多个 <code>cpp</code> |

课件决定本册的概念范围；旧卷用于练题型和深度，不预测今年题目或分值。本册的解释、状态表和答案由教学推导得到；未找到官方答案册，因此不称为官方标准答案。保留的最小例子与一般练习属于自编教学补充；涉及原材料时逐项标明“课件例题”“原题代码”或“题意转述”。

当前 L1 第 2 张写的是闭卷，E22 第 1 页写的是可带一张双面 A4 的限制性开卷，两者不能混作今年规则。六册只覆盖 C++ 基础这一段，旧卷的数据结构、算法和部分继承多态内容仍需后续学习。

## 1. 运算符重载是什么？先问为什么需要

你已知 int 的  $3 + 5$  表示整数加法。但两个自定义 Vec2 对象怎样相加，编译器不能凭类型名猜出你的意图。

```
struct Vec2 {
    int x;
    int y;
};
```

若希望二维向量逐分量相加，就要把规则写出来。**运算符重载是为适当的自定义类型提供运算符函数，让运算符表达式按定义执行操作。**

它首先仍是函数：有名字、参数类型、返回类型和函数体。M2 的参数与返回模型没有失效，只是调用语法变得更贴近操作含义。

## 2. 先实现最小的 operator+

```
Vec2 operator+(const Vec2& lhs, const Vec2& rhs) {
    return Vec2{lhs.x + rhs.x, lhs.y + rhs.y};
}
```

逐项读：返回类型 Vec2 表示提供结果对象；operator+ 是运算符函数名；lhs、rhs 是左右操作数的参数名；const Vec2& 表示只读借用来源；花括号内给出实现。

Vec2{..., ...} 用两个计算出的整数初始化一个结果 Vec2。这里不修改 lhs 或 rhs，所以结果表示新的向量值。

```
Vec2 a{1, 2};
Vec2 b{3, 4};
Vec2 c = a + b;
```

对这个已定义的非成员 operator+，可把  $a + b$  理解为调用 operator+(a, b)。计算得到  $c = \{4, 6\}$ ，a、b 内容不变。用这种读法，陌生的 operator 名字就能回到普通函数模型。

## 3. 重载能改变什么，不能改变什么？

重载定义自定义类型上的操作行为，不会改变运算符原来的优先级、结合方式和操作数数量。不能发明 \*\* 这个新运算符，也不能把 + 改成比 \* 结合更紧。

至少需要涉及类或枚举类型，不能单纯重定义两个内建 int 的加法。., ::, ?: 等也不能像 + 一样重载。

所以不要把重载想象成替换 C++ 语法规则。它是在允许的语法框架里提供类型相关操作。见 [运算符函数规则](#)。

## 4. 成员和非成员形式怎样对应？

上一节是非成员二元函数，有两个显式参数。某些运算符也可以写成成员形式：

```
struct Vec2 {
    int x;
    int y;
    Vec2 operator+(const Vec2& rhs) const {
        return Vec2{x + rhs.x, y + rhs.y};
    }
};
```

这是独立替代示例，不与前面的 `operator+` 同时放进同一练习程序，以免引入候选歧义。调用 `a + b` 时，`a` 是当前对象，`b` 对应显式参数 `rhs`，所以只写一个显式参数。末尾 `const` 表示不通过当前对象修改其普通成员。

读代码时先确定它在类内还是类外，再确定左操作数通过什么方式提供。对于 C++17，赋值运算符 `operator=` 必须是非静态成员；不能把所有运算符随意搬成同一种形式。

## 5. + 和 += 的根本差别是修改契约

对向量或字符串的通常接口：`+` 计算结果值，`+=` 修改左操作数。这是一种符合常见使用习惯的设计；具体类型仍应以其接口契约为准。

```
Vec2& operator+=(Vec2& lhs, const Vec2& rhs) {
    lhs.x += rhs.x;
    lhs.y += rhs.y;
    return lhs;
}
```

这里 `lhs` 为 `Vec2&`，可以修改调用者的左对象；`rhs` 为 `const Vec2&`，只读；返回 `Vec2&` 提供对修改后 `lhs` 的引用。

| 写法                  | 主要结果                   | 左对象是否改变           | 常见返回类型                 |
|---------------------|------------------------|-------------------|------------------------|
| <code>a + b</code>  | 计算组合值                  | 不改变               | <code>Vec2</code>      |
| <code>a += b</code> | 将组合结果写进 <code>a</code> | 改变 <code>a</code> | <code>Vec2&amp;</code> |

若 `a={1,2}`、`b={3,4}`，`Vec2 c = a+b`；使 `c={4,6}`、`a` 保持原值；随后 `a+=b`；才使 `a={4,6}`。两条语句不是“同一操作的两种排版”。

## 6. 返回引用到底让谁和谁相同？

```
Vec2 a{1, 2}, b{3, 4};
Vec2& r = (a += b);
r.x = 99;
```

`operator+=` 返回 `lhs`，而 `lhs` 引用 `a`，因此 `r` 绑定到 `a`。最后 `a.x` 为 99，`b` 未改变。这正是 M2 的引用模型在运算符上的应用。

若函数误返回一个局部结果对象的引用，函数结束后该对象消失，就会留下悬空引用。返回引用不是为了“更高级”，而是为了继续表示一个在返回后仍有效的已有对象。

要特别区分 `Vec2 r = (a += b);`：这里 `r` 是新对象，由运算结果所表示的 `a` 复制初始化；之后修改 `r` 不再等于修改 `a`。调用返回什么，与接收变量声明为什么类型，要分别看。

## 7. 链式赋值和连续 += 怎样读？

赋值运算符按右结合分组：`a = b = c` 相当于 `a = (b = c)`。常见赋值返回左对象引用，使外层继续读取已经赋值的 `b`。

`(a += b) += c` 则明确先修改 `a` 并返回 `a` 的引用，再在同一个 `a` 上追加 `c`。括号指出步骤，返回引用保证第二步的左边仍表示 `a`。

这不等于说 `a += b += c` 也按从左到右执行。它按赋值运算符的右结合结构成为 `a += (b += c)`，先改变 `b`，再把 `b` 的结果加到 `a`。返回类型、分组和状态变化要一起读。

## 8. 比较运算符也要定义“比较什么”

```
bool operator==(const Vec2& a, const Vec2& b) {
    return a.x == b.x && a.y == b.y;
}
```

返回 `bool` 表示真或假；两个 `const` 引用只读；函数体规定本例按两个分量相等来判断。自定义类型中“相等”的含义来自其定义，不能用对象地址相同替代内容相等。

对字符串，比较通常逐字符判断，关系运算使用字典序。尤其在 C++17 中，不应假定仅定义一个 `operator==` 就自动获得所有其他比较操作；按课程接口实际需要分别提供或复用实现。

## 9. 流是什么？cout 为什么是对象？

流是标准库表示输入输出序列及其状态的一类抽象。`std::cout` 是标准输出流对象；`std::cin` 是标准输入流对象。它们不是语言关键字，也不是普通函数名。

```
std::cout << 5;
```

这里 `<<` 选择适合输出流与整数的操作，把整数格式化写入输出序列。同一个符号在两个整数之间如 `3u << 1` 表示位移，在输出流上则表示输出操作；由参与类型和可用操作决定。

输出不只是“把内存原封不动扔到屏幕”：流会按类型、格式设置和状态产生字符序列。这也是字符指针为什么可输出文本而普通对象指针可显示地址的背景。

## 10. 怎样让 cout 输出自定义对象？

```
std::ostream& operator<<(std::ostream& os,
                        const Vec2& v) {
    os << '(' << v.x << ',' << v.y << ')';
    return os;
}
```

`std::ostream` 是输出流类型；`os` 为可修改的流引用，因为输出会改变流的状态；`v` 为只读对象引用；返回 `std::ostream&`，继续提供同一个流。

对 `std::cout << a`，`os` 引用 `cout`，`v` 引用 `a`。函数通过 `os` 输出分量，不应在实现里无条件改用 `cout`：传入的流还可能是文件流或字符串流。

为什么通常写成非成员？左操作数是标准库的流对象，不能为了给 `Vec2` 加输出能力，去给 `std::ostream` 类增添一个自定义成员。把函数放在自定义类型相应作用域，是自然的组合方式。

## 11. << 为什么能连续写？返回流引用是连接点

```
std::cout << a << ' ' << b;
```

`<<` 按左结合分组，可理解为 `((std::cout << a) << ' ') << b`。第一步输出 `a`，返回 `cout` 的引用；第二步继续用同一个流输出空格，再返回流；第三步输出 `b`。

这就是“一个输出流对象 + 一次操作 + 继续提供该流”。若自定义 `operator<<` 返回 `void`，后面的 `<< ' '` 就失去了可以继续输出的左操作数。

标准流已有的插入操作也采用返回流的接口形式，见 [输出流插入操作](#)。

## 12. 输入流与 >>：为什么对象参数通常不能 const?

```
std::istream& operator>>(std::istream& is, Vec2& v) {
    is >> v.x >> v.y;
    return is;
}
```

这是最小教学版，约定输入为两个整数，不处理复杂文本语法。is 引用输入流；v 是要接收数据的目标对象，所以使用 `Vec2&`，不能用 `const Vec2&` 后再给普通成员赋值。

返回 `std::istream&` 允许继续读取，也让调用者检查流状态。写 `if (std::cin >> v) { ... }` 时，表达式得到流，再按流的成功状态判断。若输入失败，不能假定所有目标成员都得到期望的新值；正式解析器可根据任务约定设计失败时的行为，本例不额外添加那套流程。

对 A1 的字符串输入接口，当前头文件注释约定读一个词，因此不等于读取整行。

## 13. 字符串流有什么用？它也是流

`<sstream>` 提供把内存字符串作为输入或输出序列的流。它可以帮助观察格式和练习输入，不必每次人工键入。

```
std::ostringstream out;
out << Vec2{3, 4};
std::string text = out.str();

std::istringstream in("8 9");
Vec2 v{0, 0};
in >> v;
```

`out.str()` 取得输出内容 `"(3,4)"`；`in` 读取后 `v` 为 `{8,9}`。同一 `operator<<` 使用传入 `os`，才能同时适用于 `cout` 和 `out`。这是检查接口可组合性的一个具体例子。

## 14. namespace 是什么，解决什么问题？

命名空间把名字组织在一个命名范围内，帮助避免不同代码的同名冲突。

```
namespace geometry {
    struct Point { int x; int y; };
}
namespace screen {
    struct Point { int row; int column; };
}
```

`geometry::Point` 与 `screen::Point` 是不同类型。`::` 指定到哪个命名空间中查找名字。命名空间不是对象，不能写 `geometry.x` 来访问成员。

`std::cout` 中 `std` 是标准库命名空间，`cout` 是其中的名字。A1 的 `mystring::str` 同理：`mystring` 是命名空间，`str` 是其中定义的类型。

## 15. using 到底省略了什么？

```
using mystring::str;
str s;
```

这让当前作用域可以直接使用 `str` 这个名字，不是在创建第二个 `str` 类型，也没有把 `mystring` 里的所有名字全部搬进来。

`using namespace mystring;` 则让未限定名字查找考虑该命名空间中的名字，范围更广。初学时用明确限定或选择性 `using`，比较容易看清名字来源。不要在公共头文件里随意添加广泛的 `using namespace`，让所有包含者都受影响。

同一命名空间可以在头文件和实现文件中再次打开，名字仍属于同一命名空间。把与 `str` 相关的非成员运算符定义在 `mystring` 内，也便于编译器结合参数类型所属命名空间查找它们。

## 16. .h 与 .cpp：接口和实现分别放什么？

**头文件通常提供供别的源文件使用的声明；源文件提供相应实现。** `.h`、`.cpp` 是常用文件命名约定，关键在内容和构建方式。

例如头文件告诉调用者存在 `int add(int,int)`，实现文件再提供函数体。调用者编译时需要知道声明；最后链接时需要找到定义。

头文件也常包含类定义，因为调用者需要知道对象的成员布局和可用接口。不能把“头文件只有声明”当绝对规则：类定义、模板定义和合适的 `inline` 函数定义都可放在头文件。

相应地，普通非 `inline` 函数定义如果通过头文件进入多个翻译单元，可能产生重复定义问题。选择什么放哪里，应从可见声明和定义数量理解。

## 17. include 做什么？头文件保护做什么？

`#include "math_utils.h"` 是预处理指令，使该头文件内容参与当前源文件的翻译。它不会自动把 `math_utils.cpp` 编译并链接进程序。

下面使用标准的条件预处理写头文件保护：

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H
int add(int a, int b);
#endif
```

`#ifndef` 检查宏是否尚未定义；`#define` 定义宏；`#endif` 结束条件区域。第一次包含时保留内容并定义标记，同一个翻译单元再次包含时跳过，从而避免重复展开。

A1 当前使用 `#pragma once`，这是常见编译器支持的头文件保护写法；它不是标准条件预处理指令的同一种语法。头文件保护也不替代链接规则，不能靠它解决多个源文件各自带入普通函数定义的问题。

## 18. 一个完整的三文件小程序

文件 `math_utils.h`：

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H
int add(int a, int b);
#endif
```

文件 math\_utils.cpp:

```
#include "math_utils.h"
int add(int a, int b) {
    return a + b;
}
```

文件 main.cpp:

```
#include <iostream>
#include "math_utils.h"
int main() {
    std::cout << add(3, 5) << '\n';
}
```

用支持 C++17 的编译器，把两个 cpp 一起构建，例如 `clang++ -std=c++17 main.cpp math_utils.cpp -o demo`。运行输出 8。只编译 main.cpp 可能在链接时找不到 add 的定义；这与声明本身语法错误是不同阶段的问题。

头文件被包含，源文件被构建系统列入编译，再由链接步骤把所需定义结合起来。不要为了找不到定义就把实现 cpp 直接 include 到所有调用者中。

## 19. 当前 A1 接口：把每个符号读成契约

```
str operator+(const str& lhs, const str& rhs);
str& operator+=(str& lhs, const str& rhs);
std::ostream& operator<<(std::ostream& os, const str& s);
std::istream& operator>>(std::istream& is, str& s);
```

这些是当前 assignment1/string.h 中 mystring 内的真实声明。+ 返回新 str，两个输入只读；+= 修改左对象并返回它的引用；<< 写流但只读 s；>> 读流并修改 s。

头文件还分别声明 str 与 const char\* 在 + 两侧的版本。s + "!" 与 "!" + s 的参数顺序不同，不能以为只写一个固定左右类型版本就自动覆盖反过来的调用。

实现应与声明的命名空间、参数类型和返回类型一致。函数体代码写得再像，如果定义到了错误作用域，也可能不是声明所指的那个函数。

## 20. 完整程序：+、+=、流一起使用

```
#include <iostream>
#include <sstream>

struct Vec2 { int x; int y; };

Vec2 operator+(const Vec2& a, const Vec2& b) {
    return Vec2{a.x + b.x, a.y + b.y};
}
Vec2& operator+=(Vec2& a, const Vec2& b) {
    a.x += b.x;
    a.y += b.y;
    return a;
}
std::ostream& operator<<(std::ostream& os,
                        const Vec2& v) {
    return os << '(' << v.x << ',' << v.y << ')';
}

int main() {
    Vec2 a{1, 2}, b{3, 4};
    Vec2 c = a + b;
    a += b;
    std::ostringstream out;
    out << a << ' ' << c;
    std::cout << out.str() << '\n';
}
```

c 取得新结果 {4,6}; a 随后被改为 {4,6}; 它们是不同的对象。字符串流收集两次输出，最终显示 (4,6) (4,6)。相同内容并不代表同一对象。

## 回到 lecture notes：不漏掉库与流的组合部分

### A. L2 第 37、41 张：仍然是普通参数模型

课件的 Point + 例子采用 `Point operator+(Point a, const Point& b)`：a 是左对象的副本，所以可修改 a 的分量后返回它；b 是右对象的只读引用。这样与本册“两边只读引用、另建结果”的写法都可得到不修改操作数的 + 契约。课件代码展示处少了一个分号，阅读时应识别为排版/代码笔误，不能把编译失败解释成参数机制不成立。

L2 第 41 张说运算符可用自由函数或成员函数实现，这需要限定：**C++17 中 operator=、operator[]、operator()、operator-> 必须采用相应的非静态成员形式，类型转换函数也必须是成员**。不是仅仅“通常建议”把 = 写成成员。其余可重载运算符按各自规则选择适合的形式；不是所有标点都能重载。

L5 第 17 张在 Point 内写 friend 声明输出函数。friend 给予该函数访问权限，但函数仍可以是非成员；Point 的 x、y 已经 public 时，单纯读取这两个字段并不必须 friend。先看“函数属于谁”，再看“谁有访问权限”。

### B. L3 第 14-19 张的 Direction：先定义类型，再定义操作

enum 是用有限个命名值定义一个枚举类型。课件的 `enum Direction { NORTH, EAST, SOUTH, WEST }`；使四个名字依次表示 0、1、2、3。

```

Direction leftFrom(Direction dir) {
    return Direction((dir + 3) % 4);
}
Direction rightFrom(Direction dir) {
    return Direction((dir + 1) % 4);
}

```

从 NORTH 向左应到 WEST：用  $(0+3)\%4$  得 3；直接做  $(0-1)\%4$  会得到负余数，不能用来安全表示这个循环规则。外面的 `Direction(...)` 把相应整数结果转换为枚举值。四方向的循环仅在输入为这四个有效值的约定下讨论。

`Direction` 的输出函数返回 `os << directionToString(dir)`，先把枚举映射成名称文本，再沿用流输出并返回原流。它不是“给 enum 加一段屏幕打印语法”，仍是可组合的函数接口。

课件的两个 ++ 声明是：

```

Direction operator++(Direction& dir);
Direction operator++(Direction& dir, int);

```

第二个多出的 `int` 是区分后置形式的标记，普通写 `dir++` 时由语言识别这一形式；它不是让使用者填“增加多少”。前置修改后返回新枚举值；后置保存旧值、修改原变量，再返回旧值。这里课件前置返回 `Direction` 值，不能把它擅自说成返回 `Direction&`。

若要每次合法地在四方向内循环，可采用课件末尾提出的变式：把增加逻辑写成 `dir=Direction((dir+1)%4)`。课件原来的 `dir+1` 在 WEST 之后会产生枚举命名值范围以外的值；本册不把这种越界转换用作 C++17 的可移植遍历终点写法。

## C. 文件流是什么？只是把同一套读写连接到文件

L5 第 20-32 张把输入来源从键盘换为文本文件，使用 `<fstream>` 的 `ifstream/ofstream`。`open` 建立流与文件的关联；`close` 结束关联。关闭与其他程序能否同时访问同一文件的具体规则有关，但不能说所有环境一律禁止其他程序并发使用已打开文件。

```

#include <fstream>
#include <string>

void showFile(std::ostream& out) {
    std::ifstream in("input.txt");
    std::string line;
    while (std::getline(in, line)) {
        out << line << '\n';
    }
}

```

这是针对已存在且可正常打开的文件演示逐行读法，不是完整的用户界面。`std::getline(in,line)` 返回同一个流；条件根据读取后的状态决定本轮是否有可处理的数据。`line` 保存本行内容，分隔换行被提取并丢弃；输出再显式加入换行。

L5 第 25 张先展示 `while(!eof())`，第 30-31 张展示把 `getline` 放进条件的写法。采用后者：只有尝试读取以后才知道是否成功，不能先问 `eof` 就保证下一次读取一定有数据。文件打开失败时循环也不会进入，实际应用应按任务要求报告打开失败。

## D. get、>>、getline：同样输入，停止规则不同

| 操作                                 | 读取单位与空白处理        | 用途       |
|------------------------------------|------------------|----------|
| <code>in &gt;&gt; word</code>      | 格式化读一个词，通常跳过前导空白 | A1 的一词语义 |
| <code>std::getline(in,line)</code> | 读到换行或指定分隔符       | 含空格的一整行  |
| <code>in.get(ch)</code>            | 取下一个字符，包括空白      | 逐字符扫描    |

L5 第 26 张的 `get()` 无参数版返回 `int`，能表示字符值和 `eof` 标记；不要先压进 `char` 再可靠地区别所有字符与 `EOF`。带 `char&` 的 `get(ch)` 则写入 `ch` 并返回流，推荐读法为 `while(in.get(ch)) { ... }`，让成功条件控制处理。

L5 第 18 张还混合了 `noskipws`、`ws`、`skipws`：`noskipws` 关闭后续格式化读取默认跳空白；`ws` 当次消耗开头连续空白；`skipws` 恢复后续默认跳空白。它们的作用范围不同。输入依次为两个空格、A、一个空格、B 时，课件链 `in >> noskipws >> a >> ws >> b >> skipws >> c` 使 `a=' '`、`b='A'`、`c='B'`，连续输出是一个开头空格再跟 AB。

## E. 流为什么能放在 if/while 中？

L5 第 27 张用自定义 `INT` 展示 `explicit operator bool()` `const`。**类型转换函数**定义对象在某种转换语境下怎样得到另一种类型；这里返回值是布尔判断，函数名包含目标类型 `bool`，不再在前面写普通返回类型。

`while(in.get(ch))` 中得到的是输入流引用，再通过流的布尔状态判断读取是否失败。这不表示读取函数直接返回了刚才那个字符的真假；正常读到 `'\0'` 也可以是一次成功的字符读取。

## F. 接口不只是一个声明：还包括行为承诺

L3 第 23 张要求接口统一、简单、充分、通用、稳定。对应 A1：声明告诉你 + 的参数和返回类型；题面进一步告诉你 + 不改输入、超长截断、<< 不补换行。这些都属于使用者依赖的契约。

L4 第 18 张的尖括号/双引号示例应结合编译器搜索路径理解。双引号通常先在源文件附近查找，尖括号通常用于配置的包含目录；不能把 `<cstring.h>` 永远错误当成语言定理。A1 用本地 `"string.h"`，不能与标准 `<string>` 或 C 库 `<cstring>` 混淆。

T1 第 1 页还特别说明：C++ 构建使用匹配的 `g++` 或 `clang++` 驱动，`gdb` 是调试器；`gcc/clang` 即便能处理 `.cpp`，也可能没有自动完成同样的 C++ 标准库链接设置。理解 `include`、编译、链接之后，就能定位这类问题发生在哪一步。

## 练习

### 题 1：操作契约

沿用 Vec2 的非成员 +、+=。a={1,2}, b={3,4}, 先 Vec2 c = a+b;, 再 Vec2& r = (a+=b);, 最后 r.x=20;。求 a、b、c 的值。

### 题 2：流为什么返回引用？

若输出函数声明为 `void operator<<(std::ostream&, const Vec2&);`, 单次输出后为什么不能照常接 `<< ' \n'`?

### 题 3：输入参数 const

把输入运算符的 Vec2& 改成 const Vec2&, 函数体仍写 `is >> v.x >> v.y;`, 是否正确?

### 题 4：名字和对象

`mystring::str s;` 中 `mystring`、`str`、`s`、`::` 分别是什么? `s.data` 的点号与 `::` 有什么区别?

### 题 5：头文件没有实现会怎样？

`main.cpp` 包含一个有 `add` 声明的头文件, 但构建时只编译 `main.cpp`, 没有提供 `add` 的定义。为什么“编译器认识 `add`”仍不足以完成程序?

### 题 6：连续 += 的分组

使用向量 += 的通常语义, `a={1,1}`, `b={2,2}`, `c={3,3}`。执行 `a += b += c;` 后 a、b、c 分别是什么?

## 练习详细解析

### 题 1 解析

`a+b` 返回新值 `{4,6}`，因此 `c={4,6}`，`a` 暂时仍为 `{1,2}`。`a+=b` 把 `a` 改为 `{4,6}`，并返回 `a` 的引用，故 `r` 绑定到 `a`。`r.x=20` 后 `a={20,6}`，`b={3,4}`，`c={4,6}`。先看修改契约，再看接收变量是对象还是引用。

### 题 2 解析

输出链按左结合。第一步若返回 `void`，外层就没有输出流作为左操作数，无法继续选择正常流插入操作。返回 `std::ostream&` 提供的是原流，使后续输出能复用同一流。返回引用是接口的连接点，并不是把输出数据返回给屏幕。

### 题 3 解析

不正确。输入要写入 `v` 的普通成员；`const Vec2&` 不允许通过该访问方式修改它们。流参数本身也以非 `const` 引用提供，因为读取会改变输入位置和状态。不要把“引用可避免复制”作为给所有参数加 `const` 的理由。

### 题 4 解析

`mystring` 是命名空间，`str` 是其中的类型，`s` 是创建的对象，`::` 进行作用域限定。`s.data` 则在具体对象 `s` 上访问 `data` 成员。前者解决名字属于哪里，后者解决当前访问哪个对象里的哪部分。

### 题 5 解析

声明让调用表达式可以接受类型检查，定义提供可执行的函数实现。在本例 `add` 被实际调用，最终程序必须取得其合适定义；缺少实现通常在链接时报告未定义符号。头文件保护和添加 `using` 都不能补出缺失函数体。

### 题 6 解析

分组是 `a += (b += c)`。先把 `b` 改为 `{5,5}`，内层返回 `b` 的引用；再把这个结果加到 `a`，得到 `a={6,6}`。`c` 保持 `{3,3}`。若误答 `b` 不变，是把所有 `+=` 当作只改最左边 `a`；若误答 `a={3,3}`，则忽略了右结合。

## 到这里，A1 的接口已具备阅读基础

现在可从参数和返回类型判断输入输出关系，再从字符存储表示理解具体操作。下一册研究对象何时存在、何时消失，避免把有效引用和指针继续用到目标寿命之外。

## 旧卷与课件练习详细解析

### F1. E22 Q22 (第 5 页) : signature 包括什么?

原题选项依次为参数数量、参数名字、参数类型、返回类型。按本课程普通非模板函数重载的讨论，答案 **a、c**；有序参数类型列表同时确定了数量与顺序。换参数名不提供新重载，仅改变返回类型也不能重载。

```
int f(int x);
int f(int y);           // 同一个函数的再次声明
double f(double x);     // 参数类型不同，可以作为重载
// double f(int x);     // 与第一个冲突：不能仅改返回类型
```

此外，int 与 const int 的传值参数在区分函数类型时不构成两个版本；int& 与 const int& 则可构成不同的参数类型。成员函数限定、模板等还有更完整规则，本题不把“signature 只有两个词”当成所有 C++ 语境的严格定义。

### F2. E22 Q4 (第 2 页) : 流只能引用传入、引用返回吗?

题意：流复杂且不能复制，因此传流和返回流必须使用引用。按旧卷讲授的“传递同一个现有流”的场景，预期判 **True**，对应 L5 第 17 张以及 A1 的 ostream&/istream& 接口。

这样理解代码：os 引用调用者流，输出改变该流状态；返回 os 的引用让下一次 << 继续使用同一流。不复制 cout，也不构造另一个输出终点。

但原句的“必须”不能当作 C++17 的无条件定理。函数也能用流指针接收访问入口；可移动的文件流、字符串流也能按值返回新对象。例如返回 std::ostringstream{} 创建的是新流，不是在复制 cout。要把旧卷教学意图与现代 C++ 的完整能力区分开，避免把引用接口原因背错。

### F3. L3 第 19 张变式：四方向循环

使用本册的模 4 变式，初始 dir=WEST，先 Direction old=dir++;，再 Direction now=++dir;。第一步后置提供旧值 WEST，dir 变 NORTH；第二步前置把 NORTH 变 EAST，并提供 EAST。因此 old=WEST、now=EAST、dir=EAST。

不要用没有取模的课件片段在 WEST 后直接继续推算合法方向；本题明确采用循环变式。后置参数 int 只辨认形式，不表示这里额外前进了一步。

### F4. 文件与字符串流的迁移题

设 std::istream in("3 4\nhello world\n");。先执行 in >> x >> y;，再第一次 getline(in,line)，随后第二次 getline(in,line)。x、y 与两次 line 是什么？

**解析。** 格式化输入提取整数 3、4，把后面的换行留在流中；第一次 getline 立即遇到这一个换行，得到空文本；第二次才读到 "hello world"。若要在读整数后跳过剩余空白再读行，可根据实际需求处理分隔符；不要把第一次空行误诊为 getline 不支持文件/字符串流。

L5 第 35-39 张的 istream 与 ifstream 使用相同输入接口；ostringstream 与 ofstream 使用相同输出接口。一个让数据来自内存字符串，一个连接文件，流读取规则没有因此改成另一套。

### F5. A1 的输出为什么不能照抄 Point 的 endl?

L5 第 17 张的 Point 示例会输出 endl，输入时也打印提示；A1 第 2 页要求 << 只写字符、不补换行，>> 读取一个词。**契约不同，不能照搬提示与换行。** 例如 out << s << '!' 应得到连续文本。读懂参数后，还要核对题面行为。